



南開大學
Nankai University

计算机学院
并行程序设计实验报告

Lab01 体系结构相关编程实验

姓名：请在此处填写

学号：请在此处填写

专业：计算机科学与技术

2026 年 4 月 3 日

目录

1	实验概述	2
1.1	实验目的	2
1.2	研究问题与待验证假设	2
1.3	实验完成情况	2
1.4	实验思路与报告结构	2
2	实验环境	2
2.1	软硬件平台	2
2.2	测试方法	3
2.3	实验流程与数据处理	3
2.4	可复现性说明	3
3	实验设计与实现	3
3.1	矩阵与向量内积	3
3.1.1	算法设计	4
3.1.2	关键实现	4
3.1.3	规模设置说明	4
3.1.4	工作集估算	4
3.2	n 个数求和	4
3.2.1	算法设计	5
3.2.2	规模设置说明	5
3.2.3	工作集估算	5
3.3	编译与运行	6
4	实验结果与分析	6
4.1	矩阵与向量内积结果	6
4.1.1	结果分析	7
4.2	n 个数求和结果	7
4.2.1	结果分析	7
4.3	综合结论	8
5	VTune 剖析	8
5.1	矩阵题结论	8
5.2	求和题结论	9
6	Godbolt 汇编分析	9

1 实验概述

1.1 实验目的

本实验对应 Lab01「体系结构相关编程」，报告组织与实验设计遵循课程指导书与配套课件中关于实验环境、实验设计、结果呈现和 profiling 分析的要求。实验的目标是围绕 CPU cache 局部性、指令级并行和编译器优化三个主题，完成两类微基准程序的实现、验证与性能测试：

1. 矩阵与向量内积：比较逐列访问的平凡算法与按行访问的 cache 优化算法，并进一步尝试循环展开。
2. n 个数求和：比较单链式累加与多路链式累加，观察减少数据相关后对超标量执行效率的影响。
3. 对比不同编译优化等级（-O0、-O2、-O3）下的运行结果，并结合 VTune 与 Godbolt 对性能来源进行剖析。

1.2 研究问题与待验证假设

结合 Lab1 文档中的题目分析，本报告重点回答以下三个问题：

1. 对矩阵题而言，改变访存顺序是否能显著改善 cache 行为，并且这种优势是否会随着工作集增大而继续扩大？
2. 对求和题而言，多累加器与循环展开是否能有效降低累加依赖链长度，从而在 -O2/-O3 下提高超标量利用率？
3. 编译器优化、算法设计和 profiling 结果之间是否能形成一致的证据链，解释实验中的主要性能差异？

对应地，本文待验证的假设为：矩阵题的主要瓶颈来源于访存模式不匹配，求和题的主要瓶颈来源于串行依赖链，而编译器高优化等级会进一步放大这两类手工优化的收益。

1.3 实验完成情况

结合 Lab01 要求，当前已经完成的内容如下：

- 基础要求：矩阵题完成 `col_naive`、`row_cache`；求和题完成 `sum_naive`、`sum_dual`。
- 进阶要求：完成循环展开版本 `row_cache_u4` 与 `sum_unroll4`。
- 补充算法：实现树形求和 `sum_pairwise`，用于对比更多算法设计思路。
- 编译器优化对比：已经在 -O0、-O2、-O3 三种编译优化等级下完成基准测试。
- profiling 分析：已经完成矩阵题的 VTune Hotspots / Memory Access 分析、求和题的 VTune Hotspots / Microarchitecture 分析，以及两组算法的 Godbolt 汇编对比分析。

1.4 实验思路与报告结构

本实验的核心不在于设计更高渐进复杂度的算法，而在于通过尽可能简单、可控的微基准，观察体系结构因素如何影响实际运行时间。为此，本文采用“源代码改写—性能测量—profiling 验证—汇编解释”的组织方式：先在源代码层面对访存顺序或累加依赖关系进行最小幅度修改，再在相同输入规模和编译环境下比较性能，最后借助 VTune 与 Godbolt 将实验现象追溯到 cache 行为、后端执行和编译器向量化等更底层的原因。这样的结构既保证了实验结论能够落到具体代码，又避免把性能差异简单归因于“编译器更聪明”或“机器更快”。

2 实验环境

2.1 软硬件平台

根据课程要求，实验部分需要明确说明软硬件平台参数，尤其是 CPU 型号、核心数、主频、各级 cache 容量和编译选项。本实验所使用的平台信息如表1所示，其中 CPU 型号、核心数、内存和 cache 容量依据本机任务管理器与实验环境截图整理得到。

项目	配置
操作系统	Microsoft Windows 10.0.26200.8037
编译器	g++ 15.2.0 (MinGW-W64 x86_64-ucrt-posix-seh)
L ^A T _E X 编译器	XeLaTeX 2025
构建方式	make o0 / make o2 / make o3 / make vtune
CPU 型号	Intel Core Ultra 7 155H
CPU 基准频率	3.80 GHz
物理核心数 / 逻辑线程数	16 / 22
L1/L2/L3 Cache	1.6 MB / 18.0 MB / 24.0 MB
内存容量	31.5 GB
存储设备	NVMe SSD
profiling 工具	Intel VTune Profiler、Godbolt (Compiler Explorer)

表 1: 实验平台信息

2.2 测试方法

本实验采用项目中的微基准程序进行性能测试。测试程序会先完成正确性验证，再对目标算法重复执行若干次并统计总耗时。计时函数使用 `std::chrono::high_resolution_clock` 封装得到的秒级时间差，测试策略遵循“对短程序重复执行、延长计时间隔以提高测量精度”的基本原则。输出字段包括算法名、问题规模、重复次数、总耗时以及相对基础算法的加速比。

对任意算法，其加速比定义为：

$$\text{speedup} = \frac{T_{\text{baseline}}}{T_{\text{current}}} \quad (1)$$

其中，矩阵题的基准算法为 `col_naive`，求和题的基准算法为 `sum_naive`。

为了尽量保证比较公平，本实验在每组测试中保持输入数据、问题规模、编译器版本和计时框架一致，仅改变“算法写法”和“编译优化等级”两个变量。换言之，本文关注的是同一平台上不同实现之间的相对差异，而不是追求某个绝对运行时间的极限值。这种控制变量方式有助于将实验现象更明确地归因到 cache 友好性、依赖链长度以及编译器优化能力上。

2.3 实验流程与数据处理

参考以往实验资料中的做法，本次报告将实验过程拆分为“程序实现、批量执行、结果整理、图表呈现”四个阶段：

1. 源程序负责实现算法、完成正确性校验，并输出结构化测试结果；
2. 通过 `Makefile` 分别生成 `-O0`、`-O2`、`-O3` 三组可执行文件；
3. 将程序输出整理到 `RESULTS.md`，再进一步抽取为表格与折线图；
4. 在报告中同时保留“原始数值表”和“趋势图”，避免只展示终端截图或孤立数据点。

这种组织方式的优点在于：一方面能让每个结论都有对应原始数据支撑，另一方面也便于后续继续接入 VTune、Godbolt 或 CSV 脚本做更细粒度分析。

2.4 可复现性说明

为了保证结果具有可重复性，当前程序已经具备以下基础条件：

- 每种算法在正式计时前都会先进行一次正确性验证；
- 对不同规模使用固定的重复次数，以避免单次运行时间过短导致计时误差过大；
- 同一组实验中的不同算法使用相同输入数据，保证对比公平。

若后续需要进一步提高实验严谨性，可再补充固定随机种子、CPU 绑核、关闭动态睿频、导出 CSV 原始数据等措施。

3 实验设计与实现

3.1 矩阵与向量内积

设矩阵 $B \in \mathbb{R}^{n \times n}$ 、向量 $a \in \mathbb{R}^n$ ，计算目标为：

$$\text{sum}[i] = \sum_{j=0}^{n-1} B[j][i] \cdot a[j], \quad 0 \leq i < n \quad (2)$$

由于 C/C++ 中二维数组按行主序存储，若按列访问矩阵，会造成跨 cache line 的非连续读；若按行访问，则更符合空间局部性。

3.1.1 算法设计

1. **col_naive**: 固定列索引 i , 内层枚举行索引 j , 直接按数学定义实现。
2. **row_cache**: 固定行索引 j , 把当前 $a[j]$ 缓存在寄存器中, 并顺序扫描该行全部元素, 降低跨行跳跃访问带来的 cache miss。
3. **row_cache_u4**: 在 **row_cache** 基础上对内层循环做四路展开, 减少循环控制与索引更新开销。

Algorithm 1 矩阵列内积的 cache 优化思路

Input: 行主序存储矩阵 B , 向量 a , 输出数组 sum , 规模 n

Output: 输出每一列与向量 a 的内积

```

1: 将  $\text{sum}[0 \dots n-1]$  置零
2: for  $j = 0$  to  $n-1$  do
3:    $aj \leftarrow a[j]$ 
4:    $base \leftarrow j \times n$ 
5:   for  $i = 0$  to  $n-1$  do
6:      $\text{sum}[i] \leftarrow \text{sum}[i] + B[base + i] \times aj$ 
7:   end for
8: end for
9: return  $\text{sum}$ 

```

3.1.2 关键实现

矩阵题中按行访问的核心实现

```

1 void row_cache(const Matrix& b, const Vector& a, Vector& sum, std::size_t n) {
2     std::fill(sum.begin(), sum.end(), 0.0);
3     for (std::size_t j = 0; j < n; ++j) {
4         const double aj = a[j];
5         const std::size_t base = j * n;
6         for (std::size_t i = 0; i < n; ++i) {
7             sum[i] += b[base + i] * aj;
8         }
9     }
10 }

```

3.1.3 规模设置说明

矩阵题选择 $n = 128, 256, 512, 1024$ 四个规模, 考虑如下:

1. 这四组规模覆盖了从较小工作集到较大工作集的增长过程, 足以观察 cache 局部性对性能的影响是否随规模扩大而增强;
2. 随着 n 增大, 矩阵数据量按平方增长, 更容易放大“按列访问”和“按行访问”的访存差异;
3. 重复次数采用分段递减策略, 目的是在保证总耗时可接受的同时, 让不同规模下的测量仍然具有稳定性。

3.1.4 工作集估算

为了将问题规模与 cache 行为联系起来, 表2 给出了矩阵题主要数据结构的大致工作集大小。这里按双精度浮点数 8 Byte 估算, 矩阵 B 占用 $8n^2$ Byte, 向量 a 与输出数组 sum 共占用 $16n$ Byte。可以看出, 测试规模覆盖了从百 KiB 到数 MiB 的区间, 这有助于观察工作集逐步逼近私有 cache、共享 cache 乃至主存时的性能变化。

规模 n	矩阵 B	向量与结果数组	总工作集
128	128 KiB	2 KiB	130 KiB
256	512 KiB	4 KiB	516 KiB
512	2.00 MiB	8 KiB	2.01 MiB
1024	8.00 MiB	16 KiB	8.02 MiB

表 2: 矩阵题工作集规模估算

3.2 n 个数求和

求和实验关注的不是算法复杂度, 而是累加过程中的数据相关。单链式累加存在严格的串行依赖, 而多路累加可把长依赖链拆成多条较短的依赖链, 更有利于处理器发掘指令级并行。

3.2.1 算法设计

1. **sum_naive**: 使用单个累加器顺序累加, 作为基准实现。
2. **sum_dual**: 使用两个累加器分别处理偶数位与奇数位, 减少单一路径的相关长度。
3. **sum_unroll4**: 采用四个累加器并配合四路循环展开。
4. **sum_pairwise**: 使用树形规约思路, 两两相加后写回数组, 体现不同设计思路, 但会引入额外访存。

Algorithm 2 四路展开求和

Input: 输入数组 $a[0 \dots n-1]$

Output: 数组所有元素之和

```

1:  $s_0, s_1, s_2, s_3 \leftarrow 0$ 
2:  $i \leftarrow 0$ 
3: while  $i + 3 < n$  do
4:    $s_0 \leftarrow s_0 + a[i]$ 
5:    $s_1 \leftarrow s_1 + a[i + 1]$ 
6:    $s_2 \leftarrow s_2 + a[i + 2]$ 
7:    $s_3 \leftarrow s_3 + a[i + 3]$ 
8:    $i \leftarrow i + 4$ 
9: end while
10: while  $i < n$  do
11:    $s_0 \leftarrow s_0 + a[i]$ 
12:    $i \leftarrow i + 1$ 
13: end while
14: return  $(s_0 + s_1) + (s_2 + s_3)$ 

```

求和实验中的四路展开实现

```

1 double sum_unroll4(const Vector& a) {
2     double sum0 = 0.0, sum1 = 0.0, sum2 = 0.0, sum3 = 0.0;
3     std::size_t i = 0;
4     for (; i + 3 < a.size(); i += 4) {
5         sum0 += a[i];
6         sum1 += a[i + 1];
7         sum2 += a[i + 2];
8         sum3 += a[i + 3];
9     }
10    for (; i < a.size(); ++i) {
11        sum0 += a[i];
12    }
13    return (sum0 + sum1) + (sum2 + sum3);
14 }

```

3.2.2 规模设置说明

求和题选择 2^{10} 、 2^{14} 、 2^{18} 、 2^{22} 四组输入规模, 主要是为了让数据规模跨越多个数量级:

1. 小规模时, 处理器更可能受益于减少依赖链长度带来的指令级并行提升;
2. 大规模时, 数据访问的带宽与 cache 行为会逐渐成为主导因素, 可以观察展开收益是否衰减;
3. 以 2^k 形式设置规模, 也便于把实验现象与 cache 容量、对齐和规约层数联系起来分析。

3.2.3 工作集估算

对于求和题, 输入数组大小为 $8n$ Byte。表3 显示, 测试规模从 8 KiB 跨越到 32 MiB, 既包含很可能完全驻留于较低层 cache 的小数组, 也包含必须频繁访问更高层 cache 甚至主存的大数组。这样的设置有利于观察“依赖链优化收益”与“内存访问开销”之间的主导关系如何随规模改变。

规模 n	输入数组大小
2^{10}	8 KiB
2^{14}	128 KiB
2^{18}	2 MiB
2^{22}	32 MiB

表 3: 求和题工作集规模估算

3.3 编译与运行

项目提供统一的 `Makefile`，分别使用如下命令生成不同优化等级的可执行文件：

构建命令

```
1 make o0
2 make o2
3 make o3
```

对应的测试程序如下：

- `bin/matrix_dot_o0.exe`、`bin/matrix_dot_o2.exe`、`bin/matrix_dot_o3.exe`
- `bin/sum_bench_o0.exe`、`bin/sum_bench_o2.exe`、`bin/sum_bench_o3.exe`

4 实验结果与分析

4.1 矩阵与向量内积结果

矩阵题测试规模为 $n = 128, 256, 512, 1024$ ，结果如表4 所示。

优化等级	规模 n	col_naive/s	row_cache/s	row_cache_u4/s	最优加速比
-00	128	0.035911	0.032405	0.032201	1.115
-00	256	0.045614	0.038836	0.038891	1.175
-00	512	0.066748	0.038642	0.038735	1.727
-00	1024	0.232900	0.036803	0.044080	6.328
-02	128	0.005892	0.002102	0.001640	3.592
-02	256	0.005733	0.002111	0.001892	3.030
-02	512	0.006746	0.002036	0.001957	3.447
-02	1024	0.048577	0.004488	0.003531	13.758
-03	128	0.005127	0.001258	0.000972	5.274
-03	256	0.005754	0.001380	0.001369	4.203
-03	512	0.007091	0.001192	0.001243	5.951
-03	1024	0.043438	0.003575	0.003100	14.011

表 4: 矩阵列内积实验结果

图4.1 展示了 -03 下不同规模的相对加速比变化趋势。

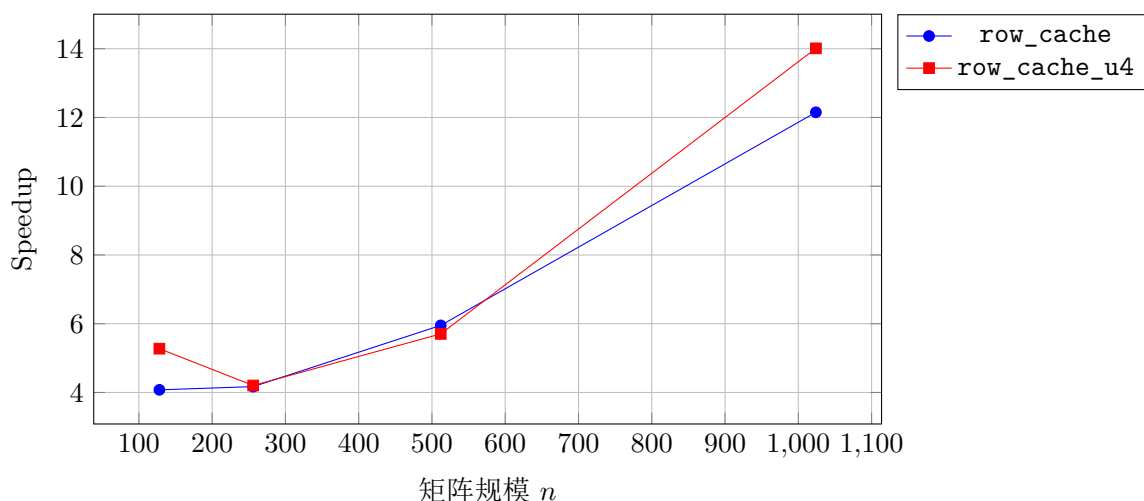


图 4.1: -03 下矩阵题相对 `col_naive` 的加速比

4.1.1 结果分析

1. `row_cache` 在所有规模和优化等级下都明显优于 `col_naive`。随着工作集从 130 KiB 增长到 8.02 MiB，按列访问的跨步读代价被持续放大，而按行顺序访问仍能利用 `cache line` 的空间局部性。
2. 图4.1 表明，矩阵题的收益会随着规模增大而增强。特别是在 `-O3`、 $n = 1024$ 时，`row_cache_u4` 的加速比达到 14.011 倍，说明“访存顺序优化 + 编译器高优化”会形成叠加效应。
3. `row_cache_u4` 大多数情况下略优于 `row_cache`，但提升幅度明显小于“按列改按行”带来的收益，因此决定性优化仍然是访存模式本身，循环展开属于次一级优化。
4. 从 `-O0` 到 `-O3` 的对比还可以看出，编译器确实能够优化地址计算、循环控制和向量化机会，但它并不能从根本上修复不合理的数据访问顺序。也就是说，矩阵题中的主要经验并不是“把优化等级开高即可”，而是应先让源代码与数据布局保持一致，再让编译器去放大这种结构优势。

4.2 n 个数求和结果

求和实验的规模分别为 2^{10} 、 2^{14} 、 2^{18} 、 2^{22} 。测试结果如表5 所示。

优化等级	规模 n	sum_naive/s	sum_dual/s	sum_unroll4/s	sum_pairwise/s	最优加速比
-O0	1024	0.041079	0.070381	0.064958	0.190056	1.000
-O0	16384	0.128563	0.223203	0.210417	0.665581	1.000
-O0	262144	0.338992	0.566687	0.474945	1.428311	1.000
-O0	4194304	0.729098	1.257167	1.137982	3.279034	1.000
-O2	1024	0.008649	0.004537	0.002311	0.006870	3.742
-O2	16384	0.039790	0.016440	0.007876	0.029359	5.052
-O2	262144	0.076637	0.045045	0.020669	0.419883	3.708
-O2	4194304	0.214646	0.164202	0.163617	1.111949	1.312
-O3	1024	0.009025	0.004388	0.002246	0.007210	4.018
-O3	16384	0.031813	0.015477	0.007777	0.029042	4.091
-O3	262144	0.095107	0.045808	0.021920	0.377787	4.339
-O3	4194304	0.190002	0.137766	0.140909	0.993874	1.379

表 5: n 个数求和实验结果

图4.2 展示了 `-O3` 下三种改进算法相对于 `sum_naive` 的加速比。

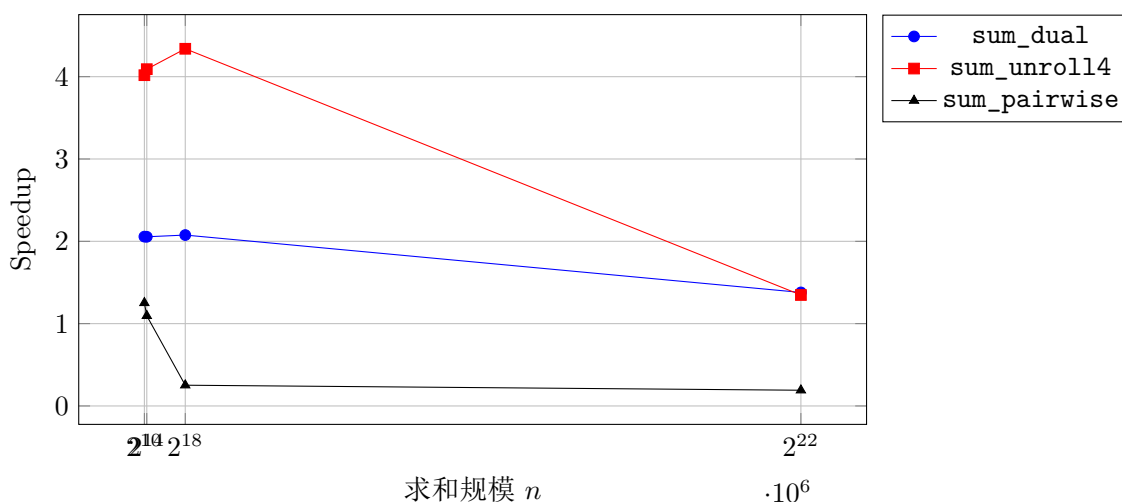


图 4.2: `-O3` 下求和题相对 `sum_naive` 的加速比

4.2.1 结果分析

1. 在 `-O0` 下，`sum_dual` 和 `sum_unroll4` 并没有获得加速，说明如果编译器几乎不做寄存器分配与向量化，多累加器结构本身也会引入额外开销。
2. 当优化等级提升到 `-O2` 和 `-O3` 后，多路累加的优势迅速显现。图4.2 显示，在 2^{10} 到 2^{18} 的范围内，`sum_unroll4` 可稳定获得约 4 倍加速，其中 `-O3`、 $n = 262144$ 时达到 4.339 倍。

- 当规模达到 2^{22} (32 MiB) 时, `sum_dual` 与 `sum_unroll4` 的优势缩小到约 1.35 倍, 说明瓶颈开始从依赖链转向更高的访存成本。
- `sum_pairwise` 在大规模下明显落后, 说明树形规约若伴随频繁写回与数据复制, 并不会优于寄存器内完成的多累加器实现。
- 这一组结果揭示了一个很典型的体系结构现象: 同样是线性时间复杂度, 实际吞吐率却会因为指令级并行暴露程度而出现显著差别。对求和这类“算术简单、重复很多”的核心循环而言, 减少单条依赖链长度往往比进一步改写外层控制逻辑更有效。

4.3 综合结论

结合两组实验, 可以得到以下结论:

- 对矩阵题而言, 访存顺序决定了 cache 行为。与数据布局一致的顺序访问能够显著降低访存延迟, 其收益会随问题规模扩大而增强。
- 对求和题而言, 算法复杂度虽然相同, 但数据相关不同。通过多路累加缩短依赖链, 可以显著提高超标量执行效率; 这一收益在中等工作集上最明显, 在超大工作集上会被访存成本部分抵消。
- 编译器优化等级会强烈影响手工优化的最终表现。高优化等级不仅能降低基线算法的执行时间, 也能把 cache 友好写法和多累加器结构进一步转化为向量化与更有效的指令调度。
- VTune 与 Godbolt 的剖析结果与性能表、趋势图能够形成一致证据链, 因此本文的主要结论不仅来自经验观察, 也获得了 profiling 和汇编层面的支持。

如果把两组实验放在一起看, 可以发现它们实际上对应两类不同但同样重要的优化思想: 矩阵题强调“让数据更容易被高效地取到”, 求和题强调“让指令更容易被并行地执行”。前者主要解决 memory hierarchy 的问题, 后者主要缓解 execution pipeline 中的数据相关问题。二者共同说明, 在现代处理器上, 程序性能往往由数据流和指令流的组织方式共同决定, 而不是仅由算法的大 O 复杂度决定。

5 VTune 剖析

受正文篇幅限制, 本节仅保留一张最有代表性的 VTune 截图, 其余 Hotspots、Memory Access 与 Microarchitecture 明细不再逐表展开, 完整截图已保存在项目目录中。之所以保留矩阵题的 Memory Access Summary, 是因为它最直接支撑“访存顺序决定 cache 行为”这一核心结论。

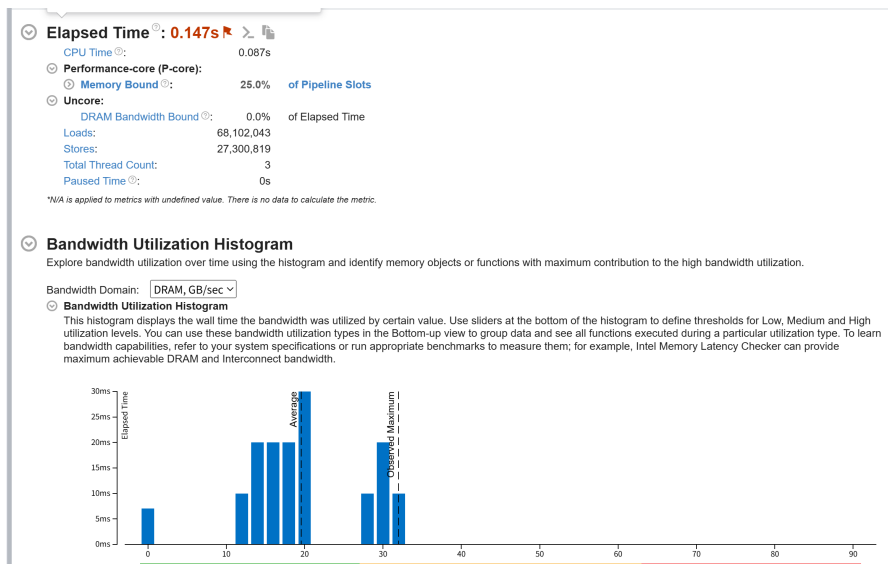


图 5.3: 受篇幅限制, 正文仅保留一张 VTune 代表性截图。该图能够直接说明矩阵题的瓶颈主要来自访问模式不友好, 而不是 DRAM 带宽打满。

5.1 矩阵题结论

在 $n = 1024$ 的 VTune 结果中, `col_naive` 的 CPU 时间约为 55.996 ms, 而 `row_cache` 与 `row_cache_u4` 分别约为 4.000 ms 和 3.000 ms; 同时整体 **Memory Bound** 为 25.0%, **DRAM Bandwidth Bound** 为 0.0%。这说明矩阵题的主要问题不是带宽饱和, 而是按列访问导致的跨步读取破坏了 cache 局部性。换言之, 处理器并不是“算不动”, 而是在等待低效的访存返回; 顺序按行访问后, 访存停顿显著下降, 与前文的加速结果一致。

5.2 求和题结论

受篇幅限制，求和题不再单独保留 VTune 截图，仅保留结论。Hotspots 结果中，`sum_unroll14` 的 CPU 时间约为 15.809 ms，明显低于 `sum_naive` 的 62.772 ms 和 `sum_dual` 的 46.375 ms；Microarchitecture Exploration 还显示 P-core 的 Back-End Bound 达到 82.1%。这说明求和题的主要瓶颈位于执行后端，多累加器确实缩短了依赖链并提升了吞吐；`sum_pairwise` 则因为中间结果写回和 `memcpy` 开销而明显变慢。VTune 因而不仅验证了“谁更快”，也解释了“为什么会更快”。

6 Godbolt 汇编分析

受正文篇幅限制，本节不逐条展示汇编截图，只保留与实验结论直接相关的观察：

1. 矩阵题中，`col_naive` 即使在 `-O3 -march=native` 下也仍然保持按列跨步访存；而 `row_cache / row_cache_u4` 更容易被编译器转化为连续加载、向量乘加和更紧凑的循环结构，因此“访问顺序改变”才是决定性优化。
2. 求和题中，`sum_naive` 始终围绕单个累加器形成长依赖链；`sum_dual` 与 `sum_unroll14` 则在源代码层面提供多条独立累加路径，更容易生成 SIMD 累加和并行调度，因此在 `-O2/-O3` 下显著更快。
3. `sum_pairwise` 的汇编包含更多中间结果写回和重复访存，说明“规约形式不同”并不天然更快；如果实现无法把主要计算保留在寄存器中，访存成本会抵消其结构优势。

从实验写作的角度看，Godbolt 的价值并不在于逐条阅读汇编指令，而在于把“源代码结构”与“机器级执行方式”对应起来。它说明本文观察到的性能差异并不是偶然噪声，而是已经在编译结果层面表现为更连续的访存、更短的依赖链和更容易触发的向量化，这使前文的性能测试与 VTune 剖析形成了闭环。